

1. [5] Considere a classe **UnsafeResourceManager**, que tem por objectivo a chamada do método **close** de um **AutoCloseable** quando não existirem mais utilizações desse objecto.

```
class UnsafeResourceManager(private val obj: AutoCloseable, usages: Int) {  
    private var currentUsages = usages  
    fun release() {  
        if (currentUsages == 0) {  
            throw IllegalStateException("usage count is already zero")  
        }  
        currentUsages -= 1  
        if (currentUsages == 0) {  
            obj.close()  
        }  
    }  
}
```

- a. [2] Realize uma versão *thread-safe* desta classe, com recurso ao uso de *locks*.
- b. [3] Realize uma versão *thread-safe* desta classe, sem recurso ao uso de *locks*.
2. [4] Realize o sincronizador **MessagePasser** com a interface apresentada em seguida.

```
class MessagePasser<T> {  
    @Throws(InterruptedException::class)  
    fun waitForMessage(timeout: Duration): T? { ... }  
    fun passToCurrentFirstWaiter(message: T): Thread? { ... }  
}
```

A função **waitForMessage** bloqueia a *thread* onde foi realizada a invocação até que uma mensagem seja enviada através da função **passToCurrentFirstWaiter**, ou até que tenha decorrido o tempo definido em **timeout**. No primeiro caso é retornada a mensagem enviada, no segundo caso é retornado **null**. A função **waitForMessage** deve ser sensível à interrupção da *thread* onde foi chamada. A função **passToCurrentFirstWaiter** deve entregar a mensagem à *thread* que espera há mais tempo, retornando a referência para essa *thread*, isto é, a *thread* em que a chamada a **waitForMessage** retornou a mensagem entregue. Caso não existam *threads* em espera aquando da chamada de **passToCurrentFirstWaiter**, esta deve retornar **null**. A mensagem passada na chamada **passToCurrentFirstWaiter** não deve ficar disponível para chamadas futuras da função **waitForMessage**.

3. [4] Realize o sincronizador **ThreadPool** com a interface apresentada em seguida

```
class ThreadPool(  
    nOfThreads: Int  
) {  
    fun <R> execute(callable: Callable<R>): CompletableFuture<R> { ... }  
}
```

Este *thread pool* tem um número fixo de *threads*, criadas aquando da construção do *pool* e usadas para a execução do trabalho agendado. As *threads* que pretendem executar funções através do *thread pool* invocam o método **execute**, especificando a função a executar com o argumento **callable**. Este método retorna imediatamente um **CompletableFuture** que se irá completar com o resultado da chamada de **callable** numa das *threads* do *thread pool*. Pode usar a classe **CompletableFuture** na resolução desta questão. Não é permitido usar um *thread pool* já existente. Lembra-se que a interface **Callable<R>** em Java tem o método

```
R call() throws Exception;
```

4. [4] Realize a classe **Box** com a interface apresentada em seguida

```
class Box<T: Any> {  
    suspend fun read(): T {...}  
    suspend fun set(value: T): Unit {...}  
    suspend fun clear(): Unit {...}  
}
```

Instâncias desta classe contém internamente um valor do tipo **T**? (i.e. valor do tipo **T** ou **null**). A função **read** suspende a execução da corrotina onde foi chamada até que o valor interno não seja **null**, retornando esse valor. A função **clear** coloca o valor interno com **null**. A função **set** coloca o valor interno com o valor passado como argumento, garantindo que todas as chamadas a **read** suspensas nesse momento vão retornar esse valor. Após a chamada da função **set**, o valor passado fica disponível para futuras chamadas da função **read**, até que o valor seja alterado através de chamadas às funções **set** ou **clear**. Nenhuma das funções deve suportar cancelamento.

5. [3] Implemente a função:

```
suspend fun <T> anyN(n: Int, futures: List<CompletableFuture<T>>): List<T>
```

que, dada uma lista não vazia de *futures* e um inteiro **n**, em que $0 < n \leq \text{futures.size}$, termina:

- Com sucesso, assim que existirem **n futures** da lista completados com sucesso. O valor retornado será uma lista com exatamente **n** elementos, contendo os valores dos *futures* completados.
- Com excepção, se todos os *futures* da lista se completarem e não existirem pelo menos **n futures** completados com sucesso. A excepção lançada pode ser qualquer uma das que foram detectadas.

Duração: 3 horas
ISEL, 18 de julho de 2023